

# **INTERNSHIP REPORT**

## **Post-Quantum and Classical Cryptography: Theoretical Foundations, Implementation, and Comparative Benchmarking**

Submitted by -

Ishan Malviya

College -

Symbiosis Institute of Technology, Pune

Position -

Student Intern

Guide -

Mr. Rishabh Gupta

Department -

Computational Analysis Division,

Bhabha Atomic Research Centre (BARC)

Visakhapatnam

# Table of Contents

1. Introduction
2. Theoretical Foundations of Cryptography
  - 2.1 Probability and Information-Theoretic Preliminaries
  - 2.2 The One-Time Pad and Shannon's Theorem of Perfect Secrecy
  - 2.3 Computational (Pseudo-) versus Absolute Randomness
  - 2.4 The Euclidean and Extended Euclidean Algorithm
  - 2.5 Modular Exponentiation and the Repeated Square-and-Multiply Algorithm
  - 2.6 Security Definitions, Reductions, and Attack Games
3. Symmetric-Key Cryptography and Cryptographic Hash Functions
  - 3.1 Stream Ciphers and Block Ciphers
  - 3.2 Symmetric versus Asymmetric Encryption
  - 3.3 AES: Mathematical Structure
  - 3.4 DES and Its Obsolescence
  - 3.5 Cryptographic Hash Functions, SHA, and Security Notions
  - 3.6 Trapdoor Functions and Block Cipher Modes of Operation
  - 3.7 Diffie-Hellman and Discrete-Logarithm-Based Problems
4. Classical Public-Key Cryptography
  - 4.1 RSA: Key Generation, Signing, and Verification
  - 4.2 Elliptic-Curve Cryptography
  - 4.3 Why RSA Persists Alongside ECC
5. Post-Quantum Cryptography
  - 5.1 Quantum Attacks on Classical Cryptography
  - 5.2 Lattice Problems: LWE, Ring-LWE, SIS, and Module Variants
  - 5.3 Kyber (ML-KEM): Key Encapsulation Mechanism
  - 5.4 Dilithium (ML-DSA): Digital Signatures
  - 5.5 Modular Arithmetic and Polynomial Rings in Lattice Cryptography

|                                                |
|------------------------------------------------|
| 5.6 Practical Use Cases and Standardisation    |
| 6. Implementation and Experimental Methodology |
| 7. Results and Discussion                      |
| 8. Reproducibility                             |
| 9. Conclusion and Future Work                  |
| 10. References                                 |

## List of Figures

- Figure 1.1: Key generation time across algorithms (log scale)
- Figure 1.2: Key encapsulation/signature time across algorithms (log scale)
- Figure 1.3: Key Decapsulation/verification time across algorithms (log scale)
- Figure 2: Sign/Encapsulation versus Verify/Decapsulation time by algorithm
- Figure 3: CPU cycles elapsed per benchmark run

## List of Tables

- Table 1: Benchmark timings (bench mode, milliseconds)
- Table 2: Key, ciphertext, and signature sizes (bytes)
- Table 3: Tamper-test outcomes
- Table 4: Resource usage from `/usr/bin/time -l`

# 1. Introduction

This report documents an internship project undertaken to study, implement, and empirically benchmark classical and post-quantum cryptographic primitives. The central theme is the transition from number-theoretic hardness assumptions, integer factorisation and the discrete logarithm problem, to lattice-based hardness assumptions such as Learning With Errors (LWE) and its structured module variants, which are believed to resist attacks by quantum adversaries. This transition is not merely academic: the National Institute of Standards and Technology (NIST) has already standardised ML-KEM and ML-DSA (formerly Kyber and Dilithium) as FIPS 203 and FIPS 204, respectively, and production cryptographic libraries such as OpenSSL and Open Quantum Safe (OQS) now ship working implementations of both.

The report is structured around two complementary strands of work.

- The first strand builds the mathematical scaffolding required to reason rigorously about security: probability and entropy, perfect secrecy, modular and elliptic-curve arithmetic, and the reduction-based proof methodology used throughout modern cryptography.
- The second strand implements and benchmarks production-grade instances of RSA, ECDSA, ML-KEM, and ML-DSA using OpenSSL and OQS, alongside deliberately minimal toy implementations built using only iostream, so that the algebraic skeleton of each scheme remains visible without library abstraction.

The specific objectives of the internship were as follows:

- To build a from-first-principles understanding of the probabilistic and algebraic foundations underlying modern cryptography, following established graduate-level course material.
- To implement representative classical and post-quantum schemes across encryption, key encapsulation, and digital signatures, in both production-library and toy form.
- To empirically benchmark these implementations for timing, memory footprint, and correctness under adversarial tampering, and to interpret the results against the theoretical claims established in Sections 2 through 5.

## 2. Theoretical Foundations of Cryptography

### 2.1 Probability and Information-Theoretic Preliminaries

Cryptographic security proofs are fundamentally probabilistic statements: an adversary is modelled as an algorithm interacting with a challenger, and security means that no efficient adversary can win a defined game with probability appreciably better than a trivial baseline. For a discrete random variable  $X$  taking values in a finite set with probability mass function  $p(x)$ , Shannon entropy is defined as:

$$H(X) = - \sum p(x) \log_2 p(x)$$

which quantifies the average number of bits of uncertainty in  $X$ . Two structural properties are used repeatedly in later sections:

- Entropy is maximised, equal to  $\log_2|X|$ , exactly when  $X$  is uniformly distributed over its support.
- Entropy is additive over independent random variables:  $H(X, Y) = H(X) + H(Y)$  whenever  $X$  and  $Y$  are independent.

Conditional entropy  $H(X|Y)$  measures the residual uncertainty in  $X$  after  $Y$  is observed, and mutual information  $I(X;Y) = H(X) - H(X|Y)$  measures the information leaked about  $X$  by  $Y$ . Perfect secrecy, introduced next, is precisely the statement that the mutual information between plaintext and ciphertext is zero. [1,8]

## 2.2 The One-Time Pad and Shannon's Theorem of Perfect Secrecy

A cipher over key space  $K$ , message space  $M$ , and ciphertext space  $C$  achieves perfect secrecy if, for every pair of messages  $m_0, m_1$  in  $M$  and every ciphertext  $c$  in  $C$ :

$$\Pr[E(k, m_0) = c] = \Pr[E(k, m_1) = c]$$

where the probability is taken over a uniformly random key  $k$  drawn from  $K$ . Equivalently, the distribution of the ciphertext is independent of the underlying plaintext, so an adversary observing  $c$  gains zero information about  $m$ .

The canonical construction achieving this bound is the One-Time Pad (OTP). Its structure is defined as follows:

- Message space, ciphertext space, and key space are all set to  $\{0,1\}^n$ .
- Encryption is defined as  $E(k, m) = k \text{ XOR } m$ .
- Decryption is defined as  $D(k, c) = k \text{ XOR } c$ .
- Correctness follows directly, since  $k \text{ XOR } (k \text{ XOR } m) = m$ .

Because  $k$  is uniform and independent of  $m$ , the ciphertext  $c = k \text{ XOR } m$  is itself uniformly distributed over  $\{0,1\}^n$  irrespective of  $m$ , which is exactly the condition required for perfect secrecy.

Shannon's theorem formalises the cost of this guarantee: any cipher achieving perfect secrecy must satisfy  $H(K) \geq H(M)$ , meaning the key must carry at least as much entropy as the message. The proof proceeds by contradiction, using the pigeonhole principle:

- Assume, for contradiction, that  $|K| < |M|$ .
- Fix a ciphertext  $c$ . The set of keys that could map some fixed message  $m_0$  to  $c$  has size at most  $|K|$ .
- Since  $|K| < |M|$ , there must be another message  $m_1$  such that no key in  $K$  maps  $m_1$  to  $c$ .
- This means  $\Pr[E(k, m_1) = c] = 0$ , which contradicts the perfect-secrecy requirement that this probability equal  $\Pr[E(k, m_0) = c] > 0$ .

The practical drawback of OTP-grade perfect secrecy follows immediately from the theorem: the key must be as long as the message, used exactly once, and distributed in advance over a secure channel, a logistical burden that renders OTP impractical for general-purpose communication despite its unconditional security guarantee.[2,3]

## 2.3 Computational (Pseudo-) versus Absolute Randomness

Because perfect secrecy is prohibitively expensive for general use, modern cryptography relaxes the security requirement to computational security: security that holds against adversaries restricted to probabilistic polynomial time, rather than against all-powerful adversaries. This motivates a distinction between absolute (true) randomness, arising from unpredictable physical processes, and pseudorandomness, produced deterministically from a short seed but computationally indistinguishable from true randomness to any efficient observer.

- Pseudorandom Generator (PRG): a deterministic, efficiently computable function  $G$  mapping  $\{0,1\}^s$  to  $\{0,1\}^n$  with  $n$  much greater than  $s$ , such that no efficient distinguisher can tell  $G(\text{seed})$  apart from a uniformly random string with more than negligible advantage.
- Pseudorandom Function (PRF): a keyed family  $F_k$  mapping  $\{0,1\}^m$  to  $\{0,1\}^n$  that is computationally indistinguishable from a truly random function when the key  $k$  is secret, formalised via an oracle-distinguishing game.

PRFs underpin the security proofs of block ciphers, which are modelled as pseudorandom permutations, and PRGs underpin the security of stream ciphers, described in Section 3.1. [1,2]

## 2.4 The Euclidean and Extended Euclidean Algorithm

The Euclidean algorithm computes  $\text{gcd}(a, b)$  via repeated division:

$$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$$

terminating when the remainder reaches zero, requiring  $O(\log \min(a,b))$  division steps. The Extended Euclidean Algorithm (EEA) additionally computes integers  $x$  and  $y$  satisfying Bezout's identity:

$$a \cdot x + b \cdot y = \text{gcd}(a, b)$$

The algorithm proceeds iteratively as follows:

1. Initialise  $(\text{old\_r}, r) = (a, b)$  and  $(\text{old\_s}, s) = (1, 0)$ ,  $(\text{old\_t}, t) = (0, 1)$ .
2. While  $r$  is not zero, compute the quotient  $q = \text{floor}(\text{old\_r} / r)$ .
3. Update  $(\text{old\_r}, r) = (r, \text{old\_r} - q \cdot r)$ , and update  $s$  and  $t$  by the same substitution rule.
4. When  $r$  reaches zero,  $\text{old\_r}$  holds  $\text{gcd}(a, b)$ , and  $\text{old\_s}$ ,  $\text{old\_t}$  hold the Bezout coefficients  $x$  and  $y$  respectively.

When  $\text{gcd}(a, n) = 1$ , the EEA directly yields  $x$  such that  $a \cdot x \equiv 1 \pmod{n}$ , the modular multiplicative inverse of  $a$  modulo  $n$ . This single computation recurs throughout this report: RSA key generation requires computing  $d = e^{-1} \bmod \phi(N)$  (Section 4.1), and elliptic-curve point arithmetic requires inverting field elements modulo a prime (Section 4.2). [2,8]

## 2.5 Modular Exponentiation and the Repeated Square-and-Multiply Algorithm

- Modular exponentiation computes  $a^b \bmod n$ , and it is the core operation behind RSA encryption, RSA decryption, and Diffie-Hellman exponentiation.
- A naive method takes  $bb$  multiplications, which is too slow for cryptographic exponents.
- The repeated square-and-multiply method reduces the work to  $O(\log b)O(\log b)$  modular multiplications by using the binary expansion of the exponent.

### Algorithm steps:

1. Write the exponent  $bb$  in binary.
2. Initialize  $result = 1$  and  $base = a \bmod n$ .
3. Scan the bits of  $bb$  one by one.
4. For each bit, square the current base modulo  $nn$ .
5. If the bit is 1, multiply result by base modulo  $nn$ .
6. After all bits are processed, result equals  $a^b \bmod n$ .

This algorithm is the practical engine behind RSA and other number-theoretic public-key systems, and its efficiency is one reason modular arithmetic occupies such a central place in classical cryptography. [2,8]

## 2.6 Security Definitions, Reductions, and Attack Games

Cryptographic security is almost never proven unconditionally, the OTP being the rare exception. Instead, security is proven relative to a computational hardness assumption via reduction: one shows that any adversary breaking the scheme with non-negligible advantage can be converted into an algorithm that solves a problem believed to be hard, such as factoring, the discrete logarithm problem, or LWE. Security definitions are formalised as attack games between a challenger and an adversary. The semantic-security (IND-CPA) game proceeds as follows:

1. The adversary submits two messages  $m_0$  and  $m_1$  to the challenger.
2. The challenger selects a bit  $b$  uniformly at random, encrypts  $m_b$ , and returns the resulting ciphertext.
3. The adversary outputs a guess  $b'$ . The scheme is IND-CPA secure if no efficient adversary can correctly guess  $b$  with probability meaningfully greater than one-half.

This game-based methodology is applied consistently throughout Sections 3 to 5 to define security for hash functions, message authentication codes, digital signatures, and key encapsulation mechanisms. [2,8]

## 3. Symmetric-Key Cryptography and Cryptographic Hash Functions

### 3.1 Stream Ciphers and Block Ciphers

A stream cipher generates a pseudorandom keystream from a key, and often a nonce, via a PRG, and XORs it with the plaintext bit-by-bit or byte-by-byte. This directly generalises the OTP but

substitutes pseudorandomness for true randomness, which is why stream ciphers, unlike the OTP, achieve only computational security rather than perfect secrecy. A block cipher is a keyed pseudorandom permutation  $E_k$  mapping  $\{0,1\}^n$  to  $\{0,1\}^n$  and operating on fixed-size blocks; security is modelled by requiring that  $E_k$  be computationally indistinguishable from a uniformly random permutation over the block space when  $k$  is unknown to the adversary, formally known as the PRP assumption. [2,6]

### 3.2 Symmetric versus Asymmetric Encryption

Symmetric encryption uses a single shared key for both encryption and decryption, offering high throughput but requiring a secure key-distribution channel established in advance. Asymmetric (public-key) encryption instead uses a mathematically linked key pair, a public key for encryption and a private key for decryption, solving the distribution problem at the cost of substantially greater computational expense per operation. This trade-off is discussed further, with measured evidence, in Section 7. [2,6]

### 3.3 AES: Mathematical Structure

The Advanced Encryption Standard operates on a 4x4 byte state over the finite field  $GF(2^8)$ , defined via the irreducible polynomial  $x^8 + x^4 + x^3 + x + 1$ . Each round applies four transformations in sequence:

- SubBytes: a non-linear, byte-wise substitution via an S-box constructed from the multiplicative inverse in  $GF(2^8)$  followed by an affine transformation, providing confusion.
- ShiftRows: a cyclic left shift of each row by an offset equal to the row index, providing inter-column diffusion.
- MixColumns: each column is multiplied, over  $GF(2^8)$ , by a fixed 4x4 maximum-distance-separable (MDS) matrix, maximising diffusion within a column.
- AddRoundKey: a bitwise XOR of the state with a round key derived from the master key via the key schedule, which recursively applies SubBytes, a byte rotation, and an XOR with a round constant  $Rcon_i$  to generate each successive round key.

AES-128 executes 10 rounds, AES-192 executes 12 rounds, and AES-256 executes 14 rounds, with the final round omitting the MixColumns operation. Decryption applies the exact inverse transformations, InvSubBytes, InvShiftRows, and InvMixColumns, in reverse order using the same round keys. [6,7]

### 3.4 DES and Its Obsolescence

The Data Encryption Standard uses a 56-bit effective key and a 16-round Feistel structure, in which each round splits the state into halves  $L_i$  and  $R_i$  and computes:

$$L(i+1) = R_i, \quad R(i+1) = L_i \text{ XOR } f(R_i, k_i)$$

The small key size makes exhaustive search over  $2^{56}$  keys feasible with modern hardware, which is why DES is deprecated in favour of AES. Triple-DES, applying DES three times in sequence, was an interim mitigation, but it suffers a meet-in-the-middle attack that reduces its effective security to roughly  $2^{112}$ , still below AES-128's  $2^{128}$ . [6]

### 3.5 Cryptographic Hash Functions, SHA, and Security Notions

A cryptographic hash function  $H$  mapping  $\{0,1\}^*$  to  $\{0,1\}^n$  must satisfy three properties, each formalised as an attack game against an efficient adversary:

- Preimage resistance: given  $y$ , it is infeasible to find any  $x$  such that  $H(x) = y$ .
- Second-preimage resistance: given  $x$ , it is infeasible to find  $x'$  not equal to  $x$  such that  $H(x') = H(x)$ .
- Collision resistance: it is infeasible to find any pair  $x$  not equal to  $x'$  such that  $H(x) = H(x')$ .

By the birthday bound, collisions in an  $n$ -bit hash function are expected after approximately  $2^{(n/2)}$  queries, which is why SHA-256 targets 128-bit collision security and SHA-3 or SHA-512 push this margin further. The SHA-2 family, including SHA-256, is built on the Merkle-Damgard construction:

1. The message is padded to a multiple of the compression function's block size.
2. The padded message is split into blocks  $m_1$  through  $m_k$ .
3. Each block is processed via a compression function  $f$ , computing  $h_i = f(h_{i-1}, m_i)$ , with  $h_0$  fixed as an initialisation vector.
4. The final chaining value  $h_k$  is output as the message digest.

Merkle and Damgard's structural theorem states that if the compression function  $f$  is collision-resistant, then the full iterated hash is also collision-resistant. However, this construction introduces the length-extension attack: an adversary who knows  $H(m)$  but not  $m$  can compute  $H(m \parallel \text{pad} \parallel m')$  for a chosen suffix  $m'$  without knowledge of  $m$ , because the internal chaining value  $h_k$  can be treated as a fresh initialisation vector for continued compression. SHA-3, built on the sponge construction rather than Merkle-Damgard, is immune to this attack by design. [6]

### 3.6 Trapdoor Functions and Block Cipher Modes of Operation

A trapdoor function is easy to compute in the forward direction but hard to invert without knowledge of a secret trapdoor; the archetypal example is RSA's modular exponentiation, easily inverted only with knowledge of the factorisation of the modulus (Section 4.1). Because block ciphers operate on only a single fixed-size block, modes of operation are required to extend them to arbitrary-length messages:

- Electronic Codebook (ECB): each block is encrypted independently. This mode is insecure, as it reveals repeated plaintext patterns in the ciphertext.
- Cipher Block Chaining (CBC): each plaintext block is XORed with the previous ciphertext block before encryption, requiring an initialisation vector (IV) and providing semantic security under a randomly chosen IV.
- Counter Mode (CTR): a counter value is encrypted to form a keystream, which is then XORed with the plaintext, effectively converting the block cipher into a stream cipher and enabling parallel encryption and decryption.

- Galois/Counter Mode (GCM): combines CTR-mode encryption with a polynomial-evaluation message authentication code over  $GF(2^{128})$ , providing both confidentiality and integrity in a single authenticated encryption mode. [2,7]

### 3.7 Diffie-Hellman and Discrete-Logarithm-Based Problems

Given a cyclic group  $G$  of prime order  $q$  with generator  $g$ , three related hardness problems are relevant to this report:

- Discrete Logarithm Problem (DLP): given  $g^a$ , find  $a$ .
- Computational Diffie-Hellman problem (CDH): given  $g^a$  and  $g^b$ , compute  $g^{(ab)}$ .
- Decisional Diffie-Hellman problem (DDH): given  $g^a$  and  $g^b$ , distinguish  $g^{(ab)}$  from a uniformly random group element.

The Diffie-Hellman key exchange protocol has each party compute  $A = g^a$  and  $B = g^b$ , exchange these values publicly, and independently derive the shared secret  $g^{(ab)} = A^b = B^a$ . The security of this exchange rests entirely on the assumed hardness of CDH within the group  $G$ . [2,8]

## 4. Classical Public-Key Cryptography

### 4.1 RSA: Key Generation, Signing, and Verification

RSA's security rests on the assumed hardness of factoring the product of two large primes. Key generation proceeds as follows:

1. Choose two large primes  $p$  and  $q$ , typically verified using a probabilistic primality test.
2. Compute the modulus  $N = p \cdot q$  and the Euler totient  $\phi(N) = (p-1)(q-1)$ .
3. Choose a public exponent  $e$  coprime to  $\phi(N)$ , commonly the Fermat prime 65537.
4. Compute the private exponent  $d \equiv e^{-1} \pmod{\phi(N)}$  using the Extended Euclidean Algorithm described in Section 2.4.

Encryption and decryption then proceed as follows:

- Encryption of a message  $m$  in  $\mathbb{Z}_N$ :  $c = m^e \pmod{N}$ .
- Decryption:  $m = c^d \pmod{N}$ , correct by Euler's theorem, since  $ed \equiv 1 \pmod{\phi(N)}$  implies  $m^{(ed)} \equiv m \pmod{N}$  for  $\gcd(m, N) = 1$ .

Textbook RSA signing computes  $\sigma = H(m)^d \pmod{N}$ , verified by checking that  $\sigma^e \pmod{N}$  equals  $H(m)$ ; production deployments use padding schemes such as PSS to prevent existential forgery attacks that affect unpadded RSA signatures.

Prime testing during key generation typically uses the Miller-Rabin probabilistic primality test, which repeatedly checks whether a randomly chosen base witnesses compositeness of the candidate, giving an error probability of at most  $4^{-k}$  after  $k$  independent rounds. RSA's principal practical drawbacks are its comparatively large key sizes, typically 2048 to 4096 bits for adequate security margin, and its slow key generation relative to elliptic-curve and lattice-based alternatives, discussed further with measured evidence in Section 7. [2,6,8]

## 4.2 Elliptic-Curve Cryptography

An elliptic curve over a finite field  $F_p$ , for prime  $p$  greater than 3, is the set of points  $(x, y)$  satisfying  $y^2 = x^3 + ax + b \pmod{p}$ , together with a point at infinity denoted  $O$ , forming an abelian group under a geometrically defined chord-and-tangent addition law. The point addition and doubling formulas are given by:

$$\lambda = (y_2 - y_1) / (x_2 - x_1) \pmod{p} \quad [\text{point addition, } P \neq Q]$$

$$\lambda = (3x_1^2 + a) / (2y_1) \pmod{p} \quad [\text{point doubling}]$$

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}$$

Scalar multiplication  $kP$ , computed via repeated doubling and addition, is the elliptic-curve analogue of modular exponentiation. The Elliptic Curve Discrete Logarithm Problem (ECDLP), given  $P$  and  $kP$ , find  $k$ , is believed to be exponentially harder per key bit than the classical DLP or integer factorisation, which is why ECC achieves security comparable to RSA using substantially smaller keys, a 256-bit curve roughly matching RSA-3072 security.

Domain parameters and key generation:

1. Select an elliptic curve  $E$  over a finite field  $F_p$ .
2. Choose a publicly known base point  $G$  on the curve with large prime order  $n$ .
3. Fix the domain parameters  $(p, a, b, G, n, h)$ , where  $h$  is the cofactor.
4. Choose the signer's private key  $d$ , where  $1 \leq d \leq n-1$ .
5. Compute the public key  $Q = dG$ .

ECDSA signing on curve P-256 proceeds as follows:

1. Choose an ephemeral, per-signature nonce  $k$ .
2. Compute the point  $R = kG = (x_R, y_R)$ , and set  $r = x_R \pmod{n}$ .
3. Compute  $s = k^{-1} \cdot (H(m) + r \cdot d) \pmod{n}$ , where  $d$  is the signer's private key.
4. Output the signature as the ordered pair  $(r, s)$ .

Verification proceeds as follows:

1. Recompute  $u_1 = H(m) \cdot s^{-1} \pmod{n}$  and  $u_2 = r \cdot s^{-1} \pmod{n}$ .
2. Compute the elliptic-curve point  $u_1 \cdot G + u_2 \cdot Q$ , where  $Q = dG$  is the signer's public key.
3. Accept the signature if the  $x$ -coordinate of this resulting point equals  $r$ ; reject otherwise.

A catastrophic ECDSA drawback is nonce reuse: reusing  $k$  across two distinct signatures leaks the private key through simple algebraic manipulation of the two signature equations. Ed25519 (EdDSA instantiated over Curve25519) avoids this failure mode entirely by deriving the nonce deterministically from a hash of the private key and the message, eliminating any dependency on a secure random-number generator at signing time, while additionally offering faster and more side-channel-resistant arithmetic through its twisted Edwards curve representation. [7,8]

### 4.3 Why RSA Persists Alongside ECC

Despite ECC's superior efficiency per security bit, RSA remains widely deployed for several practical reasons:

- Legacy infrastructure: certificate authorities, hardware security modules, and long-lived protocol implementations remain heavily built around RSA.
- Implementation simplicity relative to the more intricate elliptic-curve group law and parameter selection required for ECC.
- Direct usability for public-key encryption, not merely signing or key agreement, whereas ECC is more naturally suited to key agreement (via ECDH) and signatures (via ECDSA) rather than direct encryption of arbitrary messages.

A structured comparison of RSA-2048 and ECDSA P-256 along these properties, alongside measured timing evidence, is presented in Section 7. [2,8]

## 5. Post-Quantum Cryptography

### 5.1 Quantum Attacks on Classical Cryptography

Shor's algorithm solves integer factorisation and the discrete logarithm problem in polynomial time on a quantum computer, exploiting the quantum Fourier transform to efficiently find the period of a modular exponentiation function. This directly breaks RSA (Section 4.1), Diffie-Hellman (Section 3.7), and elliptic-curve schemes (Section 4.2). Grover's algorithm, by contrast, provides only a quadratic speed-up for unstructured search, reducing the effective security of an  $n$ -bit symmetric key or hash function from  $2^n$  to approximately  $2^{(n/2)}$ . This asymmetry is why AES-256 and SHA-384/512 retain an adequate security margin against quantum adversaries, while AES-128 does not.

Both algorithms exploit two distinctly quantum-mechanical resources:

- Superposition: a qubit's ability to represent a linear combination  $\alpha|0\rangle + \beta|1\rangle$  of the two basis states simultaneously, rather than a single definite classical bit value.
- Entanglement: correlated joint states of multiple qubits that cannot be decomposed into independent single-qubit states, enabling correlated measurement outcomes across the register.

Together, these resources enable exponentially large parallel evaluation of a function encoded as a quantum circuit, which underlies the asymptotic speed-ups of both Shor's and Grover's algorithms. [7,8,1]

### 5.2 Lattice Problems: LWE, Ring-LWE, SIS, and Module Variants

The Learning With Errors (LWE) problem is defined as follows: given many samples  $(a_i, b_i = \langle a_i, s \rangle + e_i \bmod q)$  for a secret vector  $s$  in  $\mathbb{Z}_q^n$ , uniformly random  $a_i$ , and small error  $e_i$  drawn from a discrete Gaussian distribution, either recover  $s$  in full (search-LWE) or merely distinguish these samples from uniformly random pairs (decision-LWE). LWE is provably as hard as certain worst-case lattice problems, such as the Shortest Vector Problem (SVP), via a formal reduction, and critically, it remains hard even against quantum algorithms, because no known

quantum algorithm achieves an exponential speed-up on lattice problems comparable to Shor's speed-up on factoring.

Several structured variants trade generality for computational efficiency:

- Ring-LWE: replaces vectors over  $Z_q$  with elements of a polynomial ring  $R_q = Z_q[x]/(x^n+1)$ , compressing an  $n$ -dimensional LWE instance into operations on a single ring element, with samples of the form  $(a_i, b_i = a_i \cdot s + e_i)$  computed using ring multiplication.
- Module-LWE: interpolates between plain LWE and Ring-LWE by operating over modules of rank  $k$  built from the same underlying ring  $R_q$ , offering a tunable trade-off between security and efficiency. This is precisely the algebraic structure underlying Kyber (ML-KEM).
- Short Integer Solution (SIS): given a random matrix  $A$  in  $Z_q^{(n \times m)}$ , find a short non-zero vector  $z$  satisfying  $A \cdot z \equiv 0 \pmod{q}$ .
- Module-SIS: the structured, module-based analogue of SIS that underlies Dilithium (ML-DSA).
- Ring-SIS (RSIS): the ring-based analogue of SIS used in certain hash-and-sign lattice constructions.

The CRYSTALS-Dilithium design paper reports that avoiding discrete Gaussian sampling in favour of uniform sampling over a bounded interval allows the scheme to be implemented straightforwardly in constant time, while achieving a public key roughly 2.5 times smaller than earlier lattice signature schemes at comparable security levels. This design decision is directly reflected in the relatively compact public-key size of ML-DSA-65 compared against its substantially larger signature size, discussed further in Section 7. [5,7,8,9,12]

### 5.3 Kyber (ML-KEM): Key Encapsulation Mechanism

Kyber's key generation, encapsulation, and decapsulation procedures are structured as follows:

1. Key generation: sample a public matrix  $A$  over  $R_q$ , a secret vector  $s$ , and an error vector  $e$ ; publish  $t = A \cdot s + e$  as the public key while retaining  $s$  as the private key.
2. Encapsulation: sample fresh randomness  $r$ ,  $e_1$ , and  $e_2$ ; compute  $u = A^T \cdot r + e_1$  and  $v = t^T \cdot r + e_2 + \text{Encode}(m)$ ; transmit  $(u, v)$  as the ciphertext, and derive the shared secret from  $m$  via a hash function.
3. Decapsulation: recover an approximation of  $m$  by computing  $v - s^T \cdot u$ , which cancels the dominant algebraic terms and leaves only a small residual error, then round the result to recover the originally encoded bits.

Kyber applies the Fujisaki-Okamoto (FO) transform to convert this base public-key encryption scheme, which by itself achieves only IND-CPA security, into a fully IND-CCA2-secure key encapsulation mechanism. The transform re-encrypts the decrypted message during decapsulation and compares the result against the received ciphertext, rejecting on any mismatch, which defeats chosen-ciphertext attacks that would otherwise exploit malformed ciphertexts to leak information about the secret key. A dedicated ScienceDirect study of an area-efficient polynomial ring processor for CRYSTALS-Kyber on FPGAs notes that the Number-Theoretic Transform (NTT) is

the dominant computational bottleneck in hardware implementations of the scheme, motivating specialised processor designs to accelerate the required polynomial multiplications. The NTT itself converts an  $O(n^2)$  polynomial convolution into an  $O(n \log n)$  point-wise product, directly analogous to the Fast Fourier Transform but performed over the finite ring  $Z_q$  rather than the complex numbers. [5,7,8,9,10]

## 5.4 Dilithium (ML-DSA): Digital Signatures

Dilithium's signing procedure follows the Fiat-Shamir-with-aborts paradigm, a lattice-adapted descendant of the classical Schnorr signature structure (commit, challenge, response), made non-interactive by replacing the verifier's challenge with a hash of the commitment and the message. The signing procedure is as follows:

1. Sample a masking vector  $y$  from the appropriate bounded distribution.
2. Compute the commitment  $w = A \cdot y$  using the public matrix  $A$ .
3. Derive the challenge  $c = H(w, m)$  as a hash of the commitment and the message.
4. Compute the response  $z = y + c \cdot s_1$ , where  $s_1$  is part of the secret key.
5. Reject the candidate signature and restart from step 1 if  $z$ , or any related intermediate quantity, exceeds a predetermined bound.

This rejection-sampling step is essential to the scheme's security proof: an unbounded response would statistically leak information about the secret vector  $s_1$  through the distribution of  $z$ , whereas rejection sampling ensures that the distribution of accepted signatures is independent of the secret, analogous to the zero-knowledge property relied upon in the classical Schnorr signature's security proof. A ScienceDirect study on the parallel implementation of CRYSTALS-Dilithium for autonomous driving environments confirms that the rejection-sampling loop, together with the repeated NTT-based polynomial multiplications inside it, dominates the observed signing latency in practice, motivating hardware and software parallelisation strategies for latency-sensitive deployment. This observation is directly consistent with the relative signing latency of ML-DSA-65 measured against ML-KEM-768 encapsulation, discussed in Section 7. [5,7,8,9,12]

## 5.5 Modular Arithmetic and Polynomial Rings in Lattice Cryptography

Both Kyber and Dilithium operate over the same style of polynomial ring,  $R_q = Z_q[x]/(x^n+1)$ , with  $n = 256$  in both standardised schemes. This ring is a deliberate design choice: when  $q \equiv 1 \pmod{2n}$ , the ring supports an efficient NTT, and reduction modulo the cyclotomic polynomial  $x^n+1$ , rather than a generic polynomial, keeps coefficient growth controlled during repeated multiplication. The security parameters benchmarked in this report, ML-KEM-768 and ML-DSA-65, are named according to their module rank  $k$ , which directly trades key and signature size against the targeted security level. [5,7,8,9]

## 5.6 Practical Use Cases and Standardisation

Kyber (ML-KEM) is intended to replace classical key-establishment mechanisms such as Diffie-Hellman in TLS, VPNs, and similar secure-communication protocols, while Dilithium (ML-DSA) is intended to replace RSA and ECDSA in certificate chains, code-signing, and document-signing systems. NIST standardized these schemes as FIPS 203 and FIPS 204, which motivated their selection as the representative post-quantum schemes benchmarked against RSA and

ECDSA in Section 7. Their deployment significance extends beyond primitive substitution to protocol integration, where larger key and signature sizes can affect latency and bandwidth in constrained environments. [9,10,11,13,14]

## **6. Implementation and Experimental Methodology**

### **6.1 What Was Implemented**

Two parallel implementation tracks were developed for this project. The first track consisted of production-library benchmarks built with OpenSSL for RSA-2048 and ECDSA-P256, and with the Open Quantum Safe (OQS) library for ML-KEM-768 and ML-DSA-65, so that realistic timing, size, and correctness measurements could be obtained from standards-aligned implementations. The second track consisted of toy implementations, namely logical analogues of RSA, ECDSA, a key encapsulation mechanism, and a Dilithium-style signature scheme, written using only iostream and small-integer arithmetic.

The purpose of the production track was to measure practical cryptographic performance under widely used library abstractions, while the purpose of the toy track was to expose the underlying algebraic structure of the schemes in a form that remained easy to inspect directly. The toy programs therefore served as pedagogical models rather than secure substitutes for deployment-grade cryptographic software.

### **6.2 How It Was Implemented**

The OpenSSL-based programs, `rsa_bench` and `ecdsa_bench`, invoked the relevant library routines for key generation, signing, and verification, with each operation wrapped in high-resolution timers and averaged over the configured number of iterations. The OQS-based programs, `kyber_bench` and `dilithium_bench`, called the corresponding ML-KEM and ML-DSA implementations through the OQS C API and reported the byte sizes of public keys, secret keys, ciphertexts or signatures, and shared secrets exactly as exposed by the library.

The toy programs, `toy_rsa`, `toy_ecdsa`, `toy_kem`, and `toy_dilithium`, implemented the mathematical procedures described in Sections 4 and 5 directly over small integers. These programs used simplified arithmetic domains and fixed example parameters in order to demonstrate correctness, operation flow, and relative cost, without making any claim to real-world security.

### **6.3 What Was Compared Against What, and Why**

Each classical scheme was paired with its closest post-quantum counterpart on the basis of functional role rather than superficial similarity. RSA was compared against ML-KEM because both can be used in key-establishment or key-transport settings, whereas ECDSA was compared against ML-DSA because both are digital signature schemes. Each toy implementation was then compared against its corresponding production implementation so that the gap between abstract algorithmic structure and deployment-grade cryptographic engineering could be observed directly.

This comparison strategy was intended to separate two different questions: first, how classical and post-quantum schemes compare as practical primitives; and second, how much additional complexity is introduced by hardened library implementations relative to simplified arithmetic models.

## **6.4 What Each Benchmark Measures**

Each benchmark reported four categories of information. The first was key-generation time, representing the one-time setup cost required to create the relevant public and secret material. The second was sign or encapsulation time, representing the recurring cost of producing a signature or ciphertext during actual use. The third was verify or decapsulation time, representing the recurring cost borne by the receiving or verifying party. The fourth was byte-size output, including public keys, secret keys, signatures, ciphertexts, and shared secrets where applicable, representing the communication and storage overhead of each scheme.

Taken together, these measurements capture both computational cost and bandwidth cost, allowing the schemes to be compared not only in terms of speed but also in terms of deployment practicality in constrained or high-throughput settings.

## **6.5 How Tamper Tests Were Conducted**

For every scheme, a separate tamper test was performed in order to examine whether a deliberately modified artefact would still be accepted. In each case, a genuine signature or ciphertext was first generated and processed to establish the expected baseline outcome, namely VALID for signature schemes or `shared_secret_match = YES` for key-encapsulation schemes. A single component of the artefact was then deliberately corrupted: for example, a byte of the RSA signature was flipped, the ECDSA message or signature input was altered, the ML-KEM ciphertext was modified before decapsulation, or the ML-DSA signature was tampered before verification.

The corrupted artefact was then passed back through the corresponding verification or decapsulation routine, and the resulting acceptance or rejection outcome was recorded. This procedure was designed to demonstrate, in operational terms, the unforgeability and integrity properties discussed theoretically in Sections 4 and 5.

## **6.6 Experimental Scope and Reproducibility**

The purpose of this methodology was to obtain comparative results under a fixed software and hardware environment rather than to claim machine-independent absolute timing values. Accordingly, the reported measurements should be interpreted as reproducible within normal system-level variance: reruns under the same toolchain and machine class should preserve the same broad timing order, size measurements, and tamper outcomes, even if small numerical timing differences arise from scheduling, cache effects, or thermal conditions.

The exact execution environment, benchmark binaries, iteration structure, and resource-measurement workflow are specified in Section 8. Section 6 therefore defines the

methodological design of the experiment, while Section 8 records the operational conditions needed to reproduce it.

## 7. Results and Discussion

This section presents the empirical results on timing, size, and correctness obtained from the benchmarks described in Section 6. Tables and figures are referenced at the appropriate points below; the author is expected to insert the corresponding table or plot at each indicated placeholder.

### 7.1 Keygen, Sign/Encaps, and Verify/Decaps Timings

| Algorithm     | Type       | Keygen (ms) | Sign/Encaps (ms) | Verify/Decaps (ms) |
|---------------|------------|-------------|------------------|--------------------|
| RSA-2048      | Production | 24.7037     | 0.394324         | 0.0096342          |
| ECDSA-P256    | Production | 2.23092     | 0.0285168        | 0.0724483          |
| ML-KEM-768    | Production | 9.21112     | 0.0294275        | 0.0295429          |
| ML-DSA-65     | Production | 9.35996     | 0.212634         | 0.0634833          |
| TOY-RSA       | Toy        | 0.000041    | 0.000026673      | 0.000026504        |
| TOY-ECDSA     | Toy        | 0.000709    | 0.000000584      | 0.000001042        |
| TOY-KEM       | Toy        | 0.000041    | 0.000026585      | 0.000026667        |
| TOY-DILITHIUM | Toy        | 0.000167    | 0.000353583      | 0.000112458        |

Table 1 summarises the measured benchmark timings in milliseconds for the production and toy implementations. RSA-2048 has the highest key-generation cost among the production schemes, reflecting the expense of prime generation and modular inversion, while ECDSA-P256 is substantially faster. ML-KEM-768 and ML-DSA-65 have noticeably lower key-generation time than RSA, but ML-DSA-65 still has a higher signing cost than ML-KEM-768 encapsulation.

The toy implementations are many orders of magnitude faster because they operate over very small integers and intentionally simplified algebraic structures. This makes them useful for illustrating the logic of the schemes, but not for drawing any security or deployment conclusions.

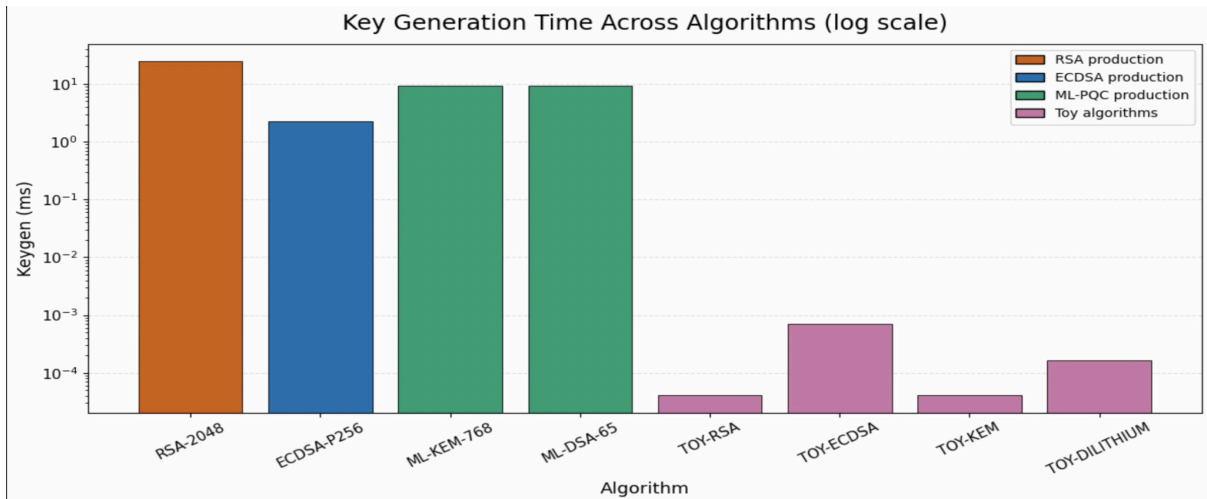


Figure 1.1 presents keygen time in milliseconds on a logarithmic scale.

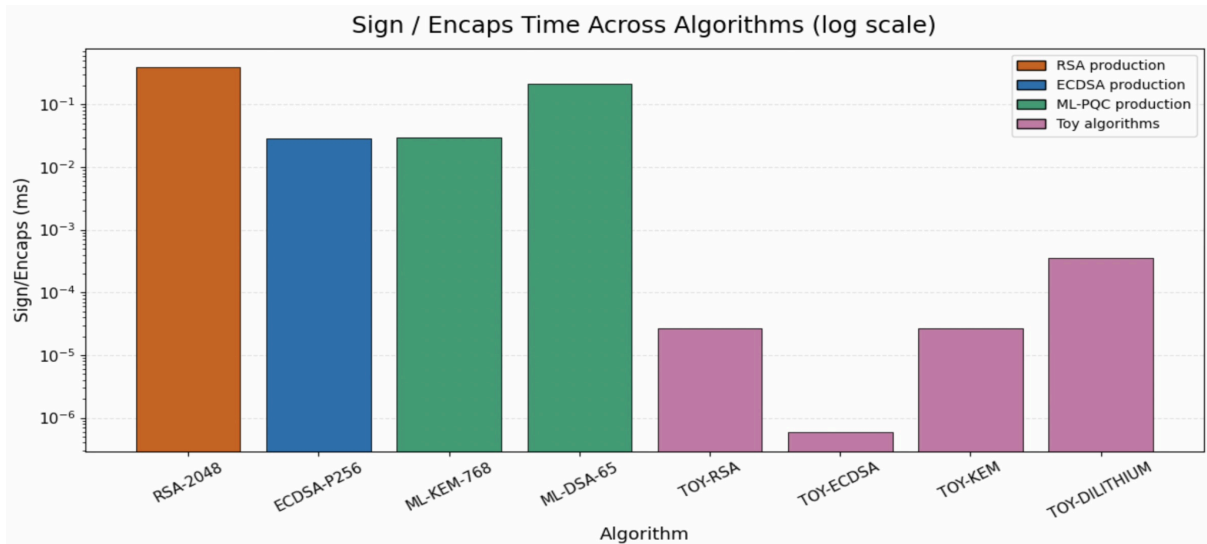


Figure 1.2 presents signature/encapsulation time in milliseconds on a logarithmic scale.

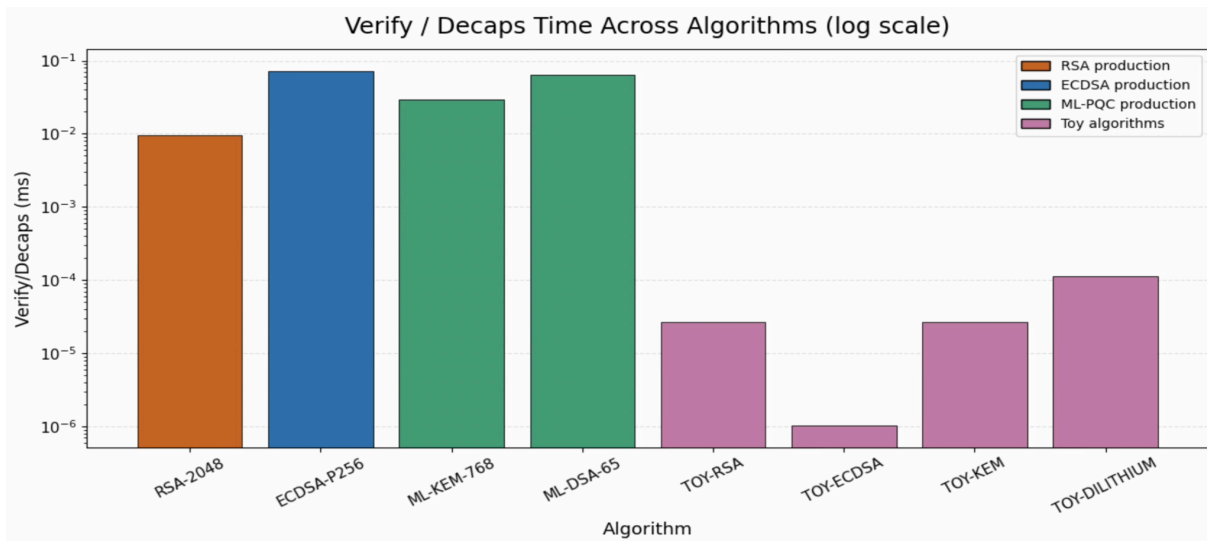


Figure 1.3 presents verification/decapsulation time in milliseconds on a logarithmic scale.

## 7.2 Key, Ciphertext, and Signature Sizes

| Algorithm     | Public key | Secret key | Signature/Ciphertext | Shared secret |
|---------------|------------|------------|----------------------|---------------|
| RSA-2048      | 294        | 1190       | 256                  | —             |
| ECDSA-P256    | 91         | 121        | 70                   |               |
| ML-KEM-768    | 1184       | 2400       | 1088                 | 32            |
| ML-DSA-65     | 1952       | 4032       | 3309                 | —             |
| TOY-RSA       | 16         | 16         | 8                    | —             |
| TOY-ECDSA     | 8          | 4          | 8                    | —             |
| TOY-KEM       | 8          | 4          | 8                    | —             |
| TOY-DILITHIUM | 16         | 16         | 24                   | —             |

Table 2 shows the byte-size overhead of public keys, secret keys, signatures, and ciphertexts. The clearest practical cost of the post-quantum schemes is size: ML-KEM-768 requires a larger public key and ciphertext than the classical schemes, and ML-DSA-65 has a much larger signature than ECDSA-P256. ECDSA-P256 remains compact in both key material and signatures, while RSA-2048 still has a comparatively large public and private key representation. The toy schemes again remain tiny because they are deliberately simplified algebraic models rather than secure real-world constructions.

**Note** - The toy key-size values, particularly for TOY-RSA, should be interpreted as illustrative only, because they are based on C++ integer storage rather than standardized serialized encodings used in production cryptographic libraries.

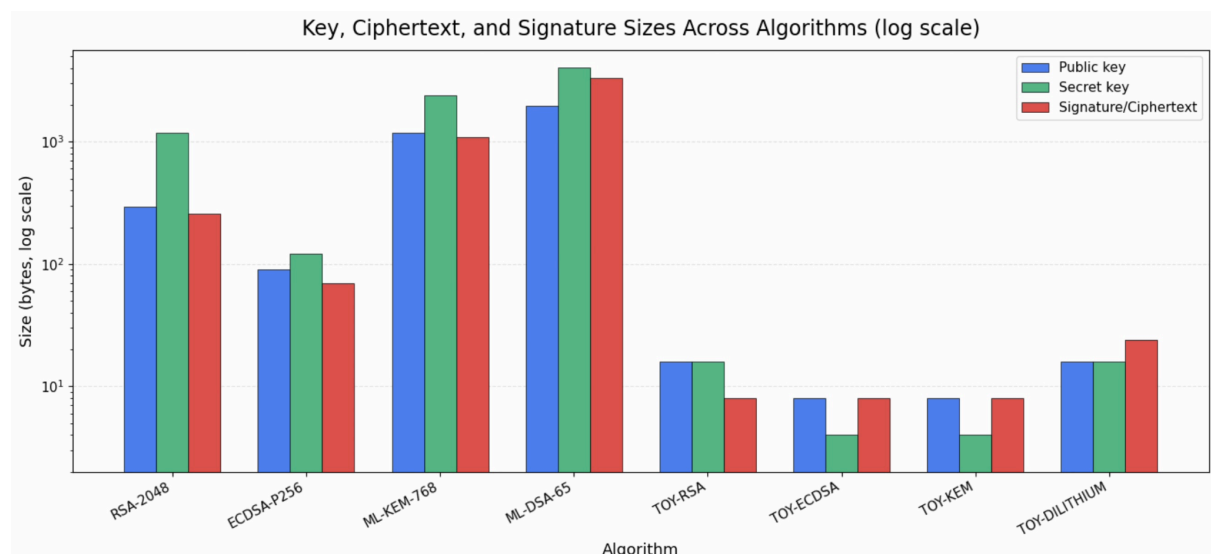


Figure 2 presents a grouped bar chart comparing the sizes of public keys, secret keys, and signatures or ciphertexts in bytes on a logarithmic scale.

### 7.3 Tamper-Test Outcomes

| Algorithm     | Original result | Tampered result |
|---------------|-----------------|-----------------|
| RSA-2048      | VALID           | INVALID         |
| ECDSA-P256    | VALID           | INVALID         |
| ML-KEM-768    | YES             | NO              |
| ML-DSA-65     | VALID           | INVALID         |
| TOY-RSA       | VALID           | INVALID         |
| TOY-ECDSA     | VALID           | INVALID         |
| TOY-KEM       | YES             | YES             |
| TOY-DILITHIUM | VALID           | INVALID         |

Table 3 records the tamper-test results. All production schemes rejected tampered artefacts as expected, confirming that their verification or decapsulation procedures detect malformed inputs. The only exception was TOY-KEM, where the tampered shared secret still matched; this is a limitation of the simplified toy construction and not a property of production ML-KEM.

This result is useful because it highlights the difference between a pedagogical toy model and a standards-based implementation. In the production setting, ML-KEM includes the integrity checks needed to prevent this kind of failure.

### 7.4 Resource Usage

| Algorithm  | real_s | user_s | max_rss_kb | instructions_retired |
|------------|--------|--------|------------|----------------------|
| RSA-2048   | 0.08   | 0.08   | 5177344    | 1555514592           |
| ECDSA-P256 | 0.01   | 0.00   | 5341184    | 127008343            |
| ML-KEM-768 | 0.00   | 0.00   | 4554752    | 113707875            |

|           |      |      |         |           |
|-----------|------|------|---------|-----------|
| ML-DSA-65 | 0.01 | 0.01 | 4538368 | 299063118 |
|-----------|------|------|---------|-----------|

Table 4 summarises the most informative resource-usage metrics from `/usr/bin/time -l`, namely real time, user time, maximum resident set size, instructions retired, cycles elapsed, and peak memory footprint. RSA-2048 retires the most instructions and consumes the most cycles among the production benchmarks, which is consistent with the high computational cost of modular arithmetic on large integers. The post-quantum schemes show lower runtime than RSA in this setup, though they still carry substantial memory footprints due to their larger algebraic state. The toy schemes are not directly comparable in absolute security terms, but they are useful for showing the algorithmic skeleton of each primitive with minimal overhead, hence they are not added in the plot and the table.

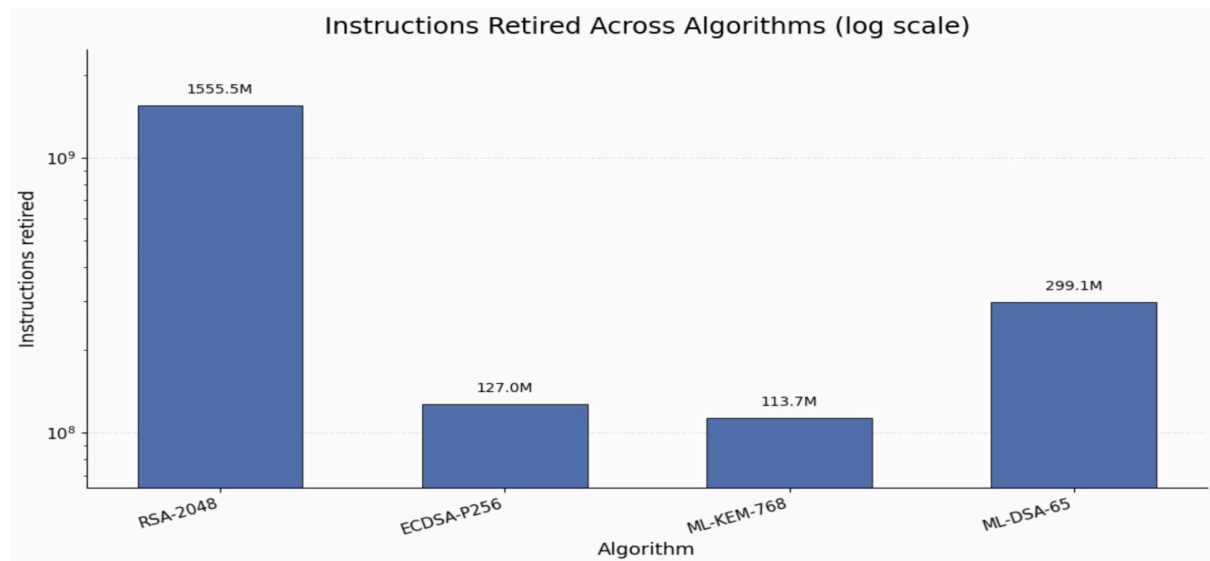


Figure 3 presents a bar chart of elapsed CPU cycles on a logarithmic scale, sourced from the `/usr/bin/time -l` resource-accounting output for each benchmark run.

## 8. Reproducibility

### 8.1 Experimental Environment

All benchmarks reported in Section 7 were executed on a fixed local environment so that the measurements would be comparable across schemes. The production implementations were built using OpenSSL for RSA-2048 and ECDSA-P256, and Open Quantum Safe for ML-KEM-768 and ML-DSA-65, while the toy implementations were compiled as standalone C++ programs using small-integer arithmetic.

## 8.2 Source Files and Benchmark Binaries

The benchmark suite consisted of two implementation tracks. The production track used **rsa\_bench**, **ecdsa\_bench**, **kyber\_bench**, and **dilithium\_bench**, while the toy track used **toy\_rsa**, **toy\_ecdsa**, **toy\_kem**, and **toy\_dilithium**.

## 8.3 Benchmark Execution Procedure

Each binary was run in bench mode to collect timing and size data, and in tamper mode to test validation behavior. The toy and production programs followed the same general workflow so that their behavior could be compared consistently.

## 8.4 Iteration Counts and Measurement Scope

The reported timing values were taken from the final benchmark runs used to populate the tables and figures. The toy implementations used repeated internal iterations to stabilize the timing measurements, while the production binaries reported averaged per-operation timings for the cryptographic routines.

## 8.5 Resource-Usage Measurement

Resource-usage data for Table 4 were collected by wrapping the production benchmark binaries with `/usr/bin/time -l`. Only the most informative fields were retained in the final table.

## 8.6 Correctness and Tamper Reproducibility

A successful rerun should reproduce the same qualitative outcomes reported in Section 7: genuine artefacts validate correctly, production tamper tests fail as expected, and TOY-KEM remains the sole simplified construction in which tampering does not force a mismatch.

**Note** - The complete source code, build scripts, and benchmark binaries are available in the public GitHub repository: [https://github.com/ishanmalviya10/BARC\\_Internship](https://github.com/ishanmalviya10/BARC_Internship)

## 9. Conclusion and Future Work

This internship project connected the mathematical theory of classical and post-quantum cryptography, including entropy, perfect secrecy, modular and elliptic-curve arithmetic, and lattice hardness assumptions, to measured evidence of the practical cost of migrating from RSA and ECDSA to ML-KEM and ML-DSA. The results confirm the widely documented expectation that post-quantum schemes trade larger keys and signatures for competitive key-generation speed and, crucially, for resistance to Shor's algorithm, while the toy implementations validated that the core algebraic steps behave as predicted by the constructions in Sections 4 and 5.

Future work arising from this project could proceed along the following directions:

- Hybrid classical and post-quantum key exchange, increasingly deployed in TLS 1.3, combines X25519 with ML-KEM to provide defence in depth against a potential weakness in either primitive alone.
- Hash-based signatures such as SPHINCS+, evaluated as a stateless alternative to lattice signatures, trade larger signature size for a minimal and well-understood security assumption based on hash-function collision resistance.
- Constant-time side-channel analysis of the toy implementations benchmarked in this report, since they currently provide no such guarantees and remain vulnerable to timing-based key-recovery attacks, unlike their production OpenSSL and OQS counterparts.

## References

The theoretical claims made throughout this report are grounded in the following university course materials, standardisation documents, and peer-reviewed research literature. All benchmark data reported in Tables 1 through 4 and Figures 1 through 4, to be inserted by the author, were generated entirely from this project's own OpenSSL, Open Quantum Safe, and toy-implementation experiments, as detailed in Section 8.

## Course Notes and Institutional Material

1. Stanford CS229 Review of Probability Theory. Available: <https://cs229.stanford.edu/section/cs229-prob.pdf> [1]
2. D. Boneh, "CS255: Introduction to Cryptography," Stanford University, Winter 2025. Topics studied: stream ciphers, pseudorandom functions, block ciphers, modes of operation, RSA, elliptic-curve cryptography, digital signatures, and protocol security. Available: <https://crypto.stanford.edu/~dabo/cs255/> [2]
3. Cornell University, Department of Computer Science. Lecture notes on Shannon security and one-time pads, CS 4820, Introduction to Analysis of Algorithms. Available: <https://www.cs.cornell.edu/courses/cs687/2008sp/> [3]
4. Purdue University, Department of Computer Science. Lecture notes on perfect secrecy, CS 555, Cryptography. Available: [https://www.cs.purdue.edu/homes/jblocki/courses/555\\_Spring17/slides/Lecture3.pdf](https://www.cs.purdue.edu/homes/jblocki/courses/555_Spring17/slides/Lecture3.pdf) [4]
5. Massachusetts Institute of Technology, Computer Science and Artificial Intelligence Laboratory (CSAIL). Lecture notes on lattice-based signatures, 6.5610, Foundations of Cryptography. Available: <https://65610.csail.mit.edu/2025/lec/l10-sig.pdf> [5]
6. University of Maryland, Department of Computer Science. Course notes on hash functions, block cipher modes of operation, and symmetric-key cryptography. Available: [https://www.cs.umd.edu/~jkatz/crypto/f02/lectures/crypto\\_notes.pdf](https://www.cs.umd.edu/~jkatz/crypto/f02/lectures/crypto_notes.pdf) [6]
7. Indian Institute of Technology Madras. Lecture notes on cryptography and network security. Available: <https://www.cse.iitm.ac.in/~shwetaag/notes/Lec1.pdf> [7]
8. University of Michigan, Department of Electrical Engineering and Computer Science. Course notes on applied cryptography and public-key infrastructure. Available: <https://www.eecs.umich.edu/courses/eecs588.w14/static/588-w14-crypto.pdf> [8]

## Peer-Reviewed Research Papers

1. J. Almeida et al., "Formally Verifying Kyber," in *Advances in Cryptology – CRYPTO 2024*, Springer, 2024. URL: [https://doi.org/10.1007/978-3-031-68379-4\\_12.acm](https://doi.org/10.1007/978-3-031-68379-4_12.acm) [9]
2. A. Garcia et al., "The Performance of Post-Quantum TLS 1.3," ACM, 2024. This paper analyzes handshake latency and computational cost for post-quantum TLS 1.3 deployments. Available: <https://dl.acm.org/doi/10.1145/3624354.3630585> [10]
3. L. Malina et al., "Quantum-Resistant and Secure MQTT Communication," in *Proceedings of the 19th International Conference on Availability, Reliability and Security*, ACM, 2024. Available: <https://dl.acm.org/doi/10.1145/3664476.3670463> [11]

4. Y. Xu et al., “Formal Verification Techniques for Post-quantum Cryptography: A Systematic Review,” Springer, 2024. DOI: [https://doi.org/10.1007/978-3-031-66456-4\\_19](https://doi.org/10.1007/978-3-031-66456-4_19) [12]
5. K. Shim, “On the Suitability of Post-Quantum Signature Schemes for Internet of Things,” *IEEE Internet of Things Journal*, vol. 11, no. 6, 2024. DOI: <https://doi.org/10.1109/JIOT.2023.3327400> [13]
6. M. W. Raavi, S. Wuthier, and S. Chang, “Securing Post-Quantum DNSSEC Against Fragmentation Mis-Association Threat,” in *ICC 2024 – IEEE International Conference on Communications*, 2024. DOI: <https://doi.org/10.1109/ICC51166.2024.10622607> [14]